

Instruktion

Dette eksamenssæt indeholder 3 opgaver, der enten skal løses i C eller C++, som angivet for hver enkelt opgave.

Hver opgave vægter i den samlede karakter med de %-satser, der står ved hver af dem. De enkelte delopgaver indgår med forskellig vægt i den samlede vurdering.

Der er i opgaverne angivet en række header- og implementeringsfiler (.c, .h, eller .cpp), der skal skrives for at løse opgaverne ved at skrive den relevante kode.

Alle disse header- og implementeringsfiler skal ved afleveringen **samles i én fil**, der afleveres i **PDF-format** på Digital Eksamen. Det skal fremgå i denne PDF-fil, hvilken opgave og fil de enkelte kode-dele kommer fra. Dvs. at kodefiler **ikke** skal afleveres enkeltvis eller som zip fil, ligesom Visual Studio solutions og projects **ikke** skal afleveres. **Al kode**, der skal bedømmes, **skal stå i denne en fil**, der vil **ikke** blive kigget på kodefiler, der vedlægges som bilag.

Der skal **ikke** inkluderes screen dumps af udskrifter fra programmet i afleveringen, men det er tilladt. I så fald kan de være i samme PDF fil, eller som bilag til afleveringen. Udskrifterne fra de kørende programmer vil først og fremmest være til fordel for dig, for at tjekke, at du har løst opgaven korrekt.

Husk angivelse af navn og studienummer på første side.

Der er 2 filer med som bilag, som kan findes på Digital Eksamen, samme sted som eksamenssættet.

Opgave 1 (35 %)

Denne opgave skal løses i C.

Følgende opgaver drejer sig om dele til et program, der kan simulere et kortspil.

Der skal bruges flere funktioner, der har med kortene at gøre. Disse samles i et modul Kortfunktioner, der består af en headerfil Kortfunktioner.h og en implementationsfil Kortfunktioner.c. Da koden således er delt op i funktioner, kan nedenstående spørgsmål løses uafhængigt af hinanden. Men de vil selvfølgelig kun køre korrekt, hvis de brugte funktioner er implementeret korrekt.

Der arbejdes kun med kortværdierne, og der tages ikke hensyn til hvor mange der er trukket allerede fra kortbunken. Der arbejdes heller ikke med sortering af kortene i en "hånd" og der tages ikke hensyn til at man fx kan få 5 Esser ved denne metode.

Kortværdierne repræsenteres med tallene fra 1 til 13, begge inkl., hvor 1 betyder Es og 13 betyder Konge efter følgende tabel:

Talværdi	Tekstværdi
1	Es
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	Knaegt
12	Dame
13	Konge

Til at udtrække et tilfældigt kort ønskes funktionen `TraekKort()`, der returnerer en tilfældig talværdi som heltal fra og med 1 til og med 13. Den har ingen parametre. Den bruger `rand()` til at finde tilfældige tal.

- a) **Skriv prototypen for `TraekKort()`, som den skal stå i en headerfil `Kortfunktioner.h` og skriv implementationen som den skal stå i en implementationsfil `Kortfunktioner.c`.**

Denne funktion skal fx bruges til at samle de 5 kort i en pokerhånd. En pokerhånd repræsenteres med et array af 5 heltal. Dette er en god test af funktionen.

- b) **Skriv `main()` til et program som det skal stå i filen `Opgave1.c`, der opretter et heltalsarray `pokerHaand` med 5 elementer. Derefter indsætter det et udtrukket kort i hvert element i arrayet ved at bruge funktionen `TraekKort()` og udskriver den indsatte værdi.**

Talværdierne er ikke så brugervenlige, så der ønskes en funktion `PrintKort`, der har én parameter, nemlig en heltalsværdi fra og med 1 til og med 13 og ingen returnværdi. Den skal udskrive en linje med den tekstværdi som står i ovenstående tabel svarende til det kort, som parameteren repræsenterer. Hvis værdien ikke er gyldig, skal den skrive en linje med teksten ”Ugyldigt kort: 25” hvor 25 erstattes med den aktuelle ugyldige værdi i udskriften.

- c) Skriv prototypen for `PrintKort()`, som den skal stå i en headerfil `Kortfunktioner.h` og skriv implementationen som den skal stå i en implementationsfil `Kortfunktioner.c`.**

Til forskellige kortspil, kan ”hænderne” være af forskellig størrelse. Til at udskrive indholdet af sådan en hånd i det generelle tilfælde ønskes en funktion `PrintHaand()`, der har 2 parametre: et heltalsarray der repræsenterer alle kortene og et heltal, der indeholder størrelsen af arrayet. Så kan funktionen bruges både til Poker, Bridge, Casino, osv.

Den skal selvfølgelig bruge funktionen `PrintKort()` for hvert kort, så udskriften bliver brugervenlig.

- d) Skriv prototypen for `PrintHaand()`, som den skal stå i en headerfil `Kortfunktioner.h` og skriv implementationen som den skal stå i en implementationsfil `Kortfunktioner.c`.**
- e) Lav en tilføjelse til `main()` fra opgave 1b) der bruger funktionen `PrintHaand` til udskrive den pokerhånd, der står i arrayet.**

Opgavesættet fortsættes på næste side!

Opgave 2 (20 %)

Denne opgave **skal** løses i C.

Der ønskes et program, der kan udregne kørselsfradraget, der indgår i skatteberegningen for personer i Danmark.

Frdraget beregnes efter følgende regler:

Afstand fra hjem til arbejde	Kørsel per dag	Beregningsmetode
Negativt tal	Ugyldigt	Ugyldig værdi, 0 kr., intet fradrag.
Fra og med 0 til og med 12 km	0 til og med 24 km	0 kr, intet fradrag
Over 12 til og med 60 km	Over 24 til og med 120 km	2,16 kr. per kilometer kørt per dag ud over de første 24
Over 60 km	Over 120 km	$(120-24)*2,16$ kr. = 207,36 kr. for de første 120 km plus 1,08 kr. per kilometer kørt per dag udover de første 120 km

Der gås ud fra at man kører ud og hjem ad samme rute på en arbejdsdag, så antallet af kørte kilometer per dag er det dobbelte af afstanden fra hjem til arbejde.

Beregninger udføres med kommatal.

Programmet skal med en passende tekst bede brugeren om at indtaste afstand fra hjem til arbejde, indlæse denne afstand som et kommatal, og udskrive det beregnede kørselsfradrag per dag med 2 decimaler ud fra ovenstående beskrivelse i en forklarende udskrift.

a) Skriv `main()` til et program som det skal stå i filen **Opgave2.c**, der gør dette.

Som testdata kan du bruge følgende inputs, som skal udskrive de angivne værdier:

Afstand til arbejdet	Kørselsfradrag per dag ved afrunding til 2 decimaler
0 km	0,00 kr
12 km	0,00 kr
12,5 km	2,16 kr
13 km	4,32 kr
59 km	203,04 kr
60 km	207,36 kr
60,5 km	208,44 kr
75 km	239,76 kr
-1 km	0,00 kr

Opgaven fortsættes på næste side!

Det er selvfølgelig ikke så godt, hvis brugeren indtaster et negativt tal, for det giver ikke mening. Derfor skal brugeren gøres opmærksom på at det er en fejl med en passende udskrift, og skal så have chancen for at indtaste en ny værdi. Dette gentages indtil der er indtastet en ikke negativ værdi.

- b) Lav ændringer til `main()` fra opgave 2a) som sikrer korrekt input fra brugeren som beskrevet.**

Opgavesættet fortsættes på næste side!

Opgave 3 (45 %)

Denne opgave **skal** løses i C++.

Denne opgave drejer sig om dele til et system, der kan administrere en kontaktiliste af personer med relevant information.

For at kunne genbruge adresseinformation andre steder i systemet, oprettes der en **Adresse** klasse, de beskrives i det følgende.

Adresse
- gadenavn_ : string - husnummer_ : int - bogstav_ : char - postnummer_ : int
+ print() : void + setGadenavn(string) : void + setHusnummer(int) : void + setBogstav(char) : void + setPostnummer(int) : void + getGadenavn() : string + getHusnummer() : int + getBogstav() : char + getPostnummer() : int

Om klassen **Adresse** gælder følgende:

Attributterne skal opfylde følgende betingelser:

Navn	Beskrivelse	Gyldige værdier	Default værdi
gadenavn_	Navnet på gaden for adressen som string	Alle værdier	"" (den tomme streng)
husnummer_	Husnummeret på gaden som heltal	Fra og med 1 til og med 999	1
bogstav_	Et evt. bogstav som et enkelt tegn, der tilføjes til Husnummer_, hvis det er nødvendigt. Hvis der ikke er en bogstavsbetegnelse, bruges ' ' (et mellemrumstegn)	' ' (et mellemrumstegn) og alle de store bogstaver fra 'A' til 'Z'	' ' (et mellemrumstegn)
postnummer_	Gadens eller byens postnummer som heltal	Fra og med 1000 til og med 9999 samt defaultværdien 0, hvis der er sket en fejl.	0

Klassen Adresse har følgende 2 parametriserede constructorer, der er ingen default constructor:

```
Adresse(  
    string gadenavn,  
    int husnummer,  
    char bogstav,  
    int postnummer);
```

Parametre:

gadenavn, husnummer, bogstav og postnummer svarer til de tilsvarende attributter.

Beskrivelse:

Parametriseret constructor, der validerer parametrene, hvor det er relevant, og initialiserer de tilsvarende attributter. Hvis en parameter ikke er gyldig, indsættes defaultværdien.

```
Adresse(  
    string gadenavn,  
    int husnummer,  
    int postnummer);
```

Parametre:

gadenavn, husnummer og postnummer svarer til de tilsvarende attributter.

Beskrivelse:

Parametriseret constructor, der validerer parametrene, hvor det er relevant, og initialiserer de tilsvarende attributter. Hvis en parameter ikke er gyldig, indsættes defaultværdien. `bogstav_` sættes til defaultværdien.

Om metoderne for klassen gælder:

```
void print() const;
```

Parametre:

Ingen

Beskrivelse:

Metoden udskriver adressens attributter i et af følgende pæne formater, alt efter om der er et bogstav tilknyttet nummeret (dvs, hvis `bogstav_` er et mellemrumstegn, skal det ikke udskrives):

Frejasgade 1, 8200

Frejasgade 2D, 8200

set-metoderne skal validere input parameteren og sætte den tilsvarende attribut. Hvis parameteren er ugyldig anvendes defaultværdien fra ovenstående tabel.

get-metoderne er almindelige metoder, der bare returnerer den tilsvarende attribut.

a) Skriv hele header-filen Adresse.h for klassen Adresse.

- b) Implementer begge constructorer for Adresse og alle klassens metoder (print(), set- og get-metoderne) i filen Adresse.cpp ud fra specifikationen ovenfor.**

Nu skal klassen testes i et program, ved at skrive en main() funktion i en ny fil.

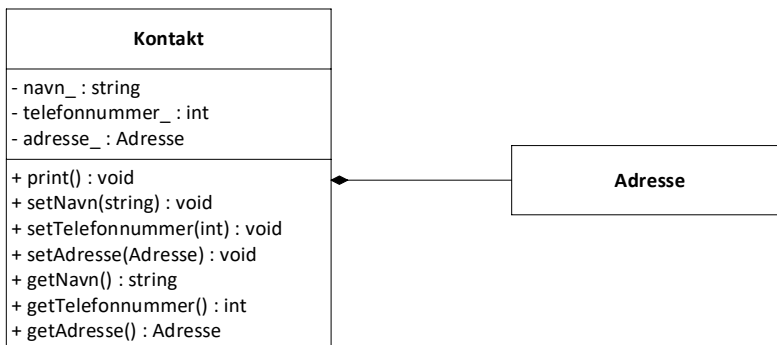
Programmet skal oprette to Adresse-objekter

- et med den parametriserede constructor med en bogstav parameter.
- et med den parametriserede constructor uden en bogstav parameter.
- Herefter udskriver programmet objekternes indhold med print().
- Brug alle set-metoderne på det andet objekt (det uden bogstav), med andre værdier end de oprindelige, sæt bl.a. adressens bogstav.
- Udskriv objektet igen efter at alle set-metoderne er blevet kaldt.

- c) Test klassen Adresse ved at skrive hele filen Opgave3.cpp med main() funktionen til et program, der udfører ovennævnte test.**

Opgaven fortsættes på næste side!
--

Klassen **Adresse** indgår i kontaktlisteret. Til at holde styr på kontaktinformationerne om en person er der oprettet en klasse **Kontakt** som benytter sig af **Adresse** klassen. Relationen mellem de to klasser ses af følgende UML-diagram. Det er selvfølgelig klassen **Adresse** fra første del af opgaven, derfor er den ikke yderligere detaljeret i dette diagram.



Kontakt er defineret og delvis implementeret i filerne **Kontakt.h** og **Kontakt.cpp**, der findes som bilag, og kan indsættes i et Visual Studio projekt. **get-** og **set-**metoderne for **Kontakt** er implementeret heri.

Om klassen **Kontakt** gælder følgende:

Attributterne skal opfylde følgende betingelser:

Navn	Beskrivelse	Gyldige værdier	Default værdi
navn_	Personens navn som en string	Ingen begrænsninger	”NN”
telefonnummer_	Personens telefonnummer som et heltal. Det antages at alle telefonnumre kan være i et 32 bit heltal.	Fra 10000000 til 99999999, samt defaultværdien 0, som betyder at der ikke er noget kendt telefonnummer	0
adresse_	Personens adresse som et Adresse objekt	Som for Adresse	Som for Adresse

Opgaven fortsættes på næste side!

Klassen Kontakt har følgende 2 parametriserede constructorer, der er ingen default constructor:

Kontakt(

**string navn,
int telefonnummer,
string gadenavn,
int husnummer,
char bogstav,
int postnummer);**

Parametre:

navn og telefonnummer svarer til de tilsvarende attributter i Kontakt.
gadenavn, husnummer, bogstav og postnummer bruges til at initialisere
memberobjektet adresse_

Beskrivelse:

Parametriseret constructor, der validerer parametrene, hvor det er relevant, og initialiserer de tilsvarende attributter. Hvis en parameter ikke er gyldig, indsættes defaultværdien.

Kontakt(

**string navn,
int telefonnummer,
string gadenavn,
int husnummer,
int postnummer);**

Parametre:

navn og telefonnummer svarer til de tilsvarende attributter i Kontakt.
gadenavn, husnummer og postnummer bruges til at initialisere memberobjektet
adresse_

Beskrivelse:

Parametriseret constructor, der validerer parametrene, hvor det er relevant, og initialiserer de tilsvarende attributter. Hvis en parameter ikke er gyldig, indsættes defaultværdien.

Opgaven fortsættes på næste side!
--

Om metoderne for klassen gælder:

void print() const;

Parametre:

Ingen

Beskrivelse:

Metoden udskriver kontaktinformation i følgende pæne format hvor de understregede data tages fra objekternes tilsvarende attributter og metoder, og er eksempler. Bemærk at formatet for adressen er det samme som for `print()` i Adresse. Understregningerne skal ikke laves i udskriften:

Kontakt: Sigurd Hansen

Telefonnummer: 23242526

Adresse:

Frejagade 1, 8200

set-metoderne skal validere input parameteren og sætte den tilsvarende attribut. Hvis parameteren er ugyldig anvendes defaultværdien fra ovenstående tabel. **Bemærk, disse er implementeret i bilaget.**

get-metoderne er almindelige metoder, der bare returnerer den tilsvarende attribut. **Bemærk, disse er implementeret i bilaget.**

d) Implementer begge constructorer for Kontakt i filen Kontakt.cpp.

e) Implementer `print()` for Kontakt i filen Kontakt.cpp.

f) Opret to Kontakt objekter i den tidligere oprettede `main()`, et med og et uden bogstav i adressen. Udskriv dem ved at kalde deres `print()` metode.